

Fundamental Problems

COMPCSI 752: Big Data Management

University of Auckland

Slides are collected and edited from

<http://webdam.inria.fr/Jorge/index9213.html>

Credit to Ninh Pham for these slides

Outline

- Web crawling
 - Near-duplicate detection
- Web search
 - PageRank
- Data cleaning
 - tfidf representation
 - Similarity join

Data cleaning

近似重复处理

- Near-duplicate detection:

- How to represent Web content efficiently for near duplicate removal?
- Context: each webpage is a text document

- d_1 The jaguar is a New World mammal of the Felidae family.
- d_2 Jaguar has designed four new engines.
- d_3 For Jaguar, Atari was keen to use a 68K family device.
- d_4 The Jacksonville Jaguars are a professional US football team.
- d_5 Mac OS X Jaguar is available at a price of US \$199 for Apple's new "family pack".
- d_6 One such ruling family to incorporate the jaguar into their name is Jaguar Paw.
- d_7 It is a big cat.

Text preprocessing

- Text preprocessing steps:
 - Number of optional steps:
 - Tokenization 标记化.
 - Stemming 词干提取
 - Stop word removal 停用词删除
 - Highly depends on the application
 - Highly depends on the document language (illustrated with English)

Tokenization

- Principle:

- Separate text into tokens (words)

标记

- Characteristics: Not so easy!

- In some languages (Chinese, Japanese), words not separated by whitespace
- Deal consistently with acronyms, numbers, units, URLs, emails...
- Compound words: *hostname*, *host-name* and *host name*. Break into two tokens or regroup them as one token? Lexicon and linguistic analysis needed!

标点

- Usually, remove punctuation and normalize case

Tokenization: Example

小句. 标点

- d_1 the₁ jaguar₂ is₃ a₄ new₅ world₆ mammal₇ of₈ the₉ felidae₁₀ family₁₁
- d_2 jaguar₁ has₂ designed₃ four₄ new₅ engines₆
- d_3 for₁ jaguar₂ atari₃ was₄ keen₅ to₆ use₇ a₈ 68k₉ family₁₀ device₁₁
- d_4 the₁ jacksonville₂ jaguars₃ are₄ a₅ professional₆ us₇ football₈ team₉
- d_5 mac₁ os₂ x₃ jaguar₄ is₅ available₆ at₇ a₈ price₉ of₁₀ us₁₁ \$199₁₂
for₁₃ apple's₁₄ new₁₅ family₁₆ pack₁₇
- d_6 one₁ such₂ ruling₃ family₄ to₅ incorporate₆ the₇ jaguar₈ into₉
their₁₀ name₁₁ is₁₂ jaguar₁₃ paw₁₄
- d_7 it₁ is₂ a₃ big₄ cat₅

Stemming 词干提取. $\rightarrow$ 形式, s, ing...

- Principle:

- Merge different forms of same word, or closely related words, into single **stem**

- Characteristics:

- Not in all applications!
- Useful for retrieving documents containing *geese* when searching for *goose*
- Various degrees of stemming
- Possibility of building different indexes, with different stemming

goose geese

Stemming: Example

- d_1 the₁ jaguar₂ is₃ a₄ new₅ world₆ mammal₇ of₈ the₉ felidae₁₀ family₁₁
- d_2 jaguar₁ has₂ designed₃ four₄ new₅ engines₆
- d_3 for₁ jaguar₂ atari₃ was₄ keen₅ to₆ use₇ a₈ 68k₉ family₁₀ device₁₁
- d_4 the₁ jacksonville₂ jaguars₃ are₄ a₅ professional₆ us₇ football₈ team₉
- d_5 mac₁ os₂ x₃ jaguar₄ is₅ available₆ at₇ a₈ price₉ of₁₀ us₁₁ \$199₁₂
for₁₃ apple's₁₄ new₁₅ family₁₆ pack₁₇
- d_6 one₁ such₂ ruling₃ family₄ to₅ incorporate₆ the₇ jaguar₈ into₉
their₁₀ name₁₁ is₁₂ jaguar₁₃ paw₁₄
- d_7 it₁ is₂ a₃ big₄ cat₅
- 



Stemming: Example

- d_1 the₁ jaguar₂ be₃ a₄ new₅ world₆ mammal₇ of₈ the₉ felidae₁₀ family₁₁
- d_2 jaguar₁ have₂ design₃ four₄ new₅ engine₆
- d_3 for₁ jaguar₂ atari₃ be₄ keen₅ to₆ use₇ a₈ 68k₉ family₁₀ device₁₁
- d_4 the₁ jacksonville₂ jaguar₃ be₄ a₅ professional₆ us₇ football₈ team₉
- d_5 mac₁ os₂ x₃ jaguar₄ be₅ available₆ at₇ a₈ price₉ of₁₀ us₁₁ \$199₁₂
for₁₃ apple₁₄ new₁₅ family₁₆ pack₁₇
- d_6 one₁ such₂ rule₃ family₄ to₅ incorporate₆ the₇ jaguar₈ into₉
their₁₀ name₁₁ be₁₂ jaguar₁₃ paw₁₄
- d_7 it₁ be₂ a₃ big₄ cat₅

<https://www.geeksforgeeks.org/python-stemming-words-with-nltk/>

Stop word removal

沒有信息的詞。

- Principle:

- Remove **uninformative** words from documents, in particular to lower the cost of storing the index

- Examples:

- determiners: a, the, this, etc.
- function verbs: be, have, make, etc.
- conjunctions: that, and, etc.
- etc.

Before removing stop words

- d_1 the₁ jaguar₂ be₃ a₄ new₅ world₆ mammal₇ of₈ the₉ felidae₁₀ family₁₁
- d_2 jaguar₁ have₂ design₃ four₄ new₅ engine₆
- d_3 for₁ jaguar₂ atari₃ be₄ keen₅ to₆ use₇ a₈ 68k₉ family₁₀ device₁₁
- d_4 the₁ jacksonville₂ jaguar₃ be₄ a₅ professional₆ us₇ football₈ team₉
- d_5 mac₁ os₂ x₃ jaguar₄ be₅ available₆ at₇ a₈ price₉ of₁₀ us₁₁ \$199₁₂
for₁₃ apple₁₄ new₁₅ family₁₆ pack₁₇
- d_6 one₁ such₂ rule₃ family₄ to₅ incorporate₆ the₇ jaguar₈ into₉
their₁₀ name₁₁ be₁₂ jaguar₁₃ paw₁₄
- d_7 it₁ be₂ a₃ big₄ cat₅

删除 ...

Stop word removal: Example

- d_1 jaguar₂ new₅ world₆ mammal₇ felidae₁₀ family₁₁
- d_2 jaguar₁ design₃ four₄ new₅ engine₆
- d_3 jaguar₂ atari₃ keen₅ 68k₉ family₁₀ device₁₁
- d_4 jacksonville₂ jaguar₃ professional₆ us₇ football₈ team₉
- d_5 mac₁ os₂ x₃ jaguar₄ available₆ price₉ us₁₁ \$199₁₂ apple₁₄
new₁₅ family₁₆ pack₁₇
- d_6 one₁ such₂ rule₃ family₄ incorporate₆ jaguar₈ their₁₀ name₁₁
jaguar₁₃ paw₁₄
- d_7 big₄ cat₅

Word occurrence information

- Text preprocessing result:
 - Each text document is represented as a set of terms (tokens) or shingles
- Define similarity measure:
 - Use set similarity measure (Jaccard similarity, containment similarity,...) to define the similarity between documents
- But... we have ignored the word occurrence information
 - D1 = “Rose is a rose is a rose is a rose” → {rose} *how many time.*
 - D2 has 1000 words where “coronavirus” appears 2 times
 - D3 has 1000 words where “coronavirus” appears 30 times

Weighted terms occurrences

- Relevance can be computed by giving a **weight** to term occurrences
 - Terms occurring **frequently** in a given **document**: more **relevant**
 - The **term frequency** $\text{tf}(\mathbf{t}, \mathbf{d})$ is the number of occurrences of a term $\mathbf{t}$ in a document $\mathbf{d}$, divided by the total number of terms in $\mathbf{d}$ (normalization)

$$\text{tf}(t, d) = \frac{n_{t,d}}{\sum_{t'} n_{t',d}}$$

出现频率
总的词数

where $\mathbf{n}_{\mathbf{t},\mathbf{d}}$ is the number of occurrences of any term $\mathbf{t}'$ in document $\mathbf{d}$

Weighted terms occurrences

- Relevance can be computed by giving a **weight** to term occurrences
 - Terms occurring **rarely** in the document **collection** as a whole: more **informative** 逆文档频率
 - The **inverse document frequency** **idf(t)** is obtained from the division of the total number of documents by the number of documents where **t** occurs:

$$\text{idf}(t) = \log \frac{|D|}{|\{d' \in D \mid n_{t,d'} > 0\}|}$$

→ 文档总数

t 至少出现一次的文档数

where the logarithmic base is 2

tfidf weighting

- Relevance can be computed by giving a **weight** to term occurrences
 - Term Frequency-Inverse Document Frequency **tfidf(t, d)** of the term **t** in document **d** is the combination of **tf(t, d)** and **idf(t)**

$$\text{tfidf}(t, d) = \frac{n_{t,d}}{\sum_{t'} n_{t',d}} \cdot \log \frac{|D|}{|\{d' \in D \mid n_{t,d'} > 0\}|}$$

$n_{t,d}$ number of occurrences of t in d
 D set of all documents

tfidf representation

- Each document in the collection is represented as a set of terms with its tfidf weight

- $D1 = \{(family, 0.13), (jaguar, 0.04), (new, 0.2), (world, 0.47)\}$
- $D2 = \{(jaguar, 0.4), (new, 0.24)\}$
- $D3 = \{(jaguar, 0.4), (football, 0.47), (us, 0.3)\}$

- Define the **inner product** as a similarity measure

相同点权重相加

$$\langle S, T \rangle = \sum_{t \text{ in both } S \text{ and } T} \text{tfidf}(t, S) * \text{tfidf}(t, T)$$

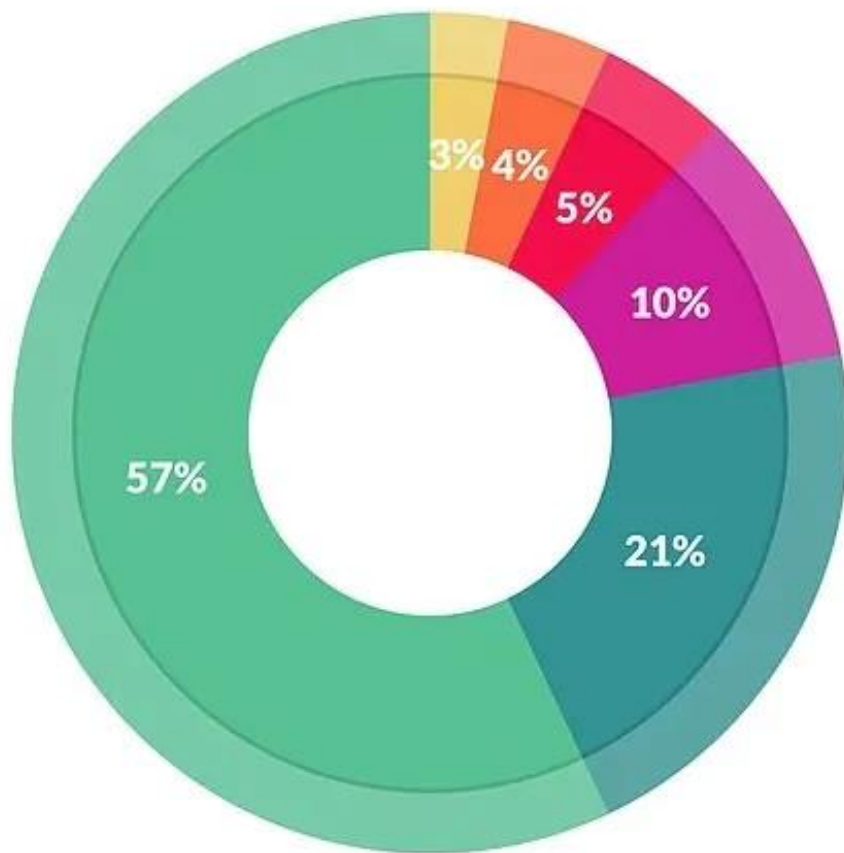
- $\langle D1, D2 \rangle = 0.04 * 0.4 + 0.2 * 0.24 = 0.064$
- $\langle D1, D3 \rangle = 0.04 * 0.4 = 0.016$

Similarity

Outline

- Web crawling
 - Near-duplicate detection
- Web search
 - PageRank
- Data cleaning
 - tfidf representation
 - Similarity join

Data cleaning – The heavy task?



What's the least enjoyable part of data science?

- Building training sets: 10%
- Cleaning and organizing data: 57%
- Collecting data sets: 21%
- Mining data for patterns: 3%
- Refining algorithms: 4%
- Other: 5%

Example

Table R

Name	Addr	Phone
Jack Lemmon	Maple St.	430-871-8294
Harrison Ford	Culver Blvd	292-918-2913
Tom Hanks	Main St.	2340762-1234
.....		

Table S

Name	Addr	Phone
Ton Hanks	Main Street	234-162-1234
Kevin Spacey	Frost Blvd	928-184-2813
Jack Lemon	Maple Street	430-817-8294
.....		

Find records from different datasets that could be the same entity.

Similarity join

- Similarity join:

- Given a set of n documents $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ in $\mathbf{R}^d$ and define the similarity measure by their inner product

$$\text{sim}(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{y} = \sum_{j=1}^d x_j y_j$$

- Given a threshold t , finding all pairs $\mathbf{x}, \mathbf{y}$ in $\mathbf{X}$ s.t. $\text{sim}(\mathbf{x}, \mathbf{y}) \geq t$

- Application:

- Plagiarism detection, collaborative filtering
- Near-duplicate detection, record linkage (entity resolution)

tfidf representation

- Collection:

- $D1 = \{(\text{jaguar}, 0.4), (\text{new}, 0.2), (\text{family}, 0.13), (\text{world}, 0.47)\}$
- $D2 = \{(\text{jaguar}, 0.4), (\text{new}, 0.24)\}$
- $D3 = \{(\text{jaguar}, 0.4), (\text{football}, 0.47), (\text{us}, 0.3)\}$
- Dim: $\{(\text{jaguar}, 1), (\text{new}, 2), (\text{family}, 3), (\text{world}, 4), (\text{football}, 5), (\text{us}, 6)\}$

- High dimensional vector representations:

- Dim: $\{(\text{jaguar}, 1), (\text{new}, 2), (\text{family}, 3), (\text{world}, 4), (\text{football}, 5), (\text{us}, 6)\}$
- $D1 = \{0.4 \quad , 0.2 \quad , 0.13 \quad , 0.47 \quad , 0 \quad , 0 \quad \}$
- $D2 = \{0.4 \quad , 0.24 \quad , 0 \quad , 0 \quad , 0 \quad , 0 \quad \}$
- $D3 = \{0.4 \quad , 0 \quad , 0 \quad , 0 \quad , 0.47 \quad , 0.3 \quad \}$

Sequential solutions

- Naïve algorithm:
 - $O(n^2)$: too expensive for all practical purposes
- Exploit data sparsity:
 - Observation: In most of data sets, vectors in $\mathbf{X}$ are sparse
 - Any two vectors $\mathbf{x}$, $\mathbf{y}$ whose non-zero coordinates do not intersect:
 $\text{sim}(\mathbf{x}, \mathbf{y}) = 0 < t$
 - Only check two vectors $\mathbf{x}$, $\mathbf{y}$ whose non-zero coordinates intersect with at least one coordinate (how to check efficiently?)

只看非零

Inverted index construction

- Inverted index:
 - For each term, create the list of documents where this term occurs

Term	Posting list containing documents
family	(d1, .13), (d3, .13), (d6, .08), (d5, .07)
football	(d4, .47)
jaguar	(d1, .04), (d2, .04), (d3, .04), (d4, .04), (d6, .04), (d5, .02)
new	(d2, .24), (d1, .20), (d5, .10)
rule	(d6, .28)
us	(d4, .30), (d5, .15)
world	(d1, .47)
...	

Inverted Index construction

- Small scale:
 - Disk storage, with **memory mapping** techniques
 - Secondary index (called vocabulary) for offset of each term in main index
 - usually a B-tree
- Large scale:
 - Distributed on a cluster of machines
 - Hashing gives the machine responsible for a given term
- Updating the index is costly, so only **batch operations** (not one-by-one addition of term occurrences)

Inverted index solution

Term

Documents

family	(d1, .13), (d3, .13), (d6, .08), (d5, .07)
football	(d4, .47)
jaguar	(d1, .04), (d2, .04), (d3, .04), (d4, .04), (d6, .04), (d5, .02)
new	(d2, .24), (d1, .20), (d5, .10)
rule	(d6, .28)
us	(d4, .30), (d5, .15)
world	(d1, .47)
...	

family: (d1, d3), (d1, d6), (d1, d5), (d3, d6), (d3, d5), (d6, d5)

...

new: (d2, d1), (d2, d5), (d1, d5)

...

Candidate
pairs

Prefix-filtering optimization

- Observation:

- We do not want to index all documents for each term. Can we reduce the index size?
- Do not want to compute $\text{sim}(\mathbf{x}, \mathbf{y})$ if
 - $\text{sim}(\mathbf{x}, \mathbf{y}) = x_1y_1 + x_2y_2 + \dots + x_ky_k + x_{k+1}y_{k+1} + \dots + x_dy_d < t$
- Define vector $\mathbf{m}$ containing the **maximum** value of each dimension/term from $\mathbf{X}$

- $x_1m_1 + x_2m_2 + \dots + x_km_k + x_{k+1}m_{k+1} + \dots + x_dm_d$

$< t$

$\geq t$

Maximum contribution
from dimensions 1 to k

→ 取每个词项
最大TF-IDF值
做后继

Prefix-filtering optimization

- Observation:

- We do not want to index all documents for each term. Can we reduce the index size?
- Do not want to compute $\text{sim}(\mathbf{x}, \mathbf{y})$ if
 - $\text{sim}(\mathbf{x}, \mathbf{y}) = x_1y_1 + x_2y_2 + \dots + x_ky_k + x_{k+1}y_{k+1} + \dots + x_dy_d < t$
- Define vector $\mathbf{m}$ containing the **maximum** value for each dimension/term from $\mathbf{X}$

$$\underbrace{x_1m_1 + x_2m_2 + \dots + x_km_k}_{\text{Maximum contribution from dimensions 1 to } k} + \underbrace{x_{k+1}m_{k+1} + \dots + x_dm_d}_{\text{If no intersection between } \mathbf{x} \text{ and } \mathbf{y} \text{ on dimensions } k+1 \text{ to } d \text{ then } \text{sim}(\mathbf{x}, \mathbf{y}) < t}$$

$< t$

$\geq t$

Maximum contribution
from dimensions 1 to k

If no intersection between
 $\mathbf{x}$ and $\mathbf{y}$ on dimensions $k+1$
to d then $\text{sim}(\mathbf{x}, \mathbf{y}) < t$

Prefix-filtering optimization

- Observation:

- We do not want to index all documents for each term. Can we reduce the index size?

football, occure once → 0.

- Prefix-filtering strategy:

- Consider the max vector $\mathbf{m}$ s.t. $\mathbf{m}_j = \max_{\mathbf{x} \in \mathbf{X}} \mathbf{x}_j$
- For each $\mathbf{x}$ in $\mathbf{X}$, find $\mathbf{p}(\mathbf{x})$ the **largest** dimension s.t. $\sum_{j \leq \mathbf{p}(\mathbf{x})} \mathbf{x}_j \mathbf{m}_j < t$
- For each $\mathbf{x}$ in $\mathbf{X}$, define signature $\mathbf{S}(\mathbf{x})$ the coordinates after $\mathbf{p}(\mathbf{x})$

$$\mathbf{S}(\mathbf{x}) = \{ \mathbf{x}_j \mid j > \mathbf{p}(\mathbf{x}) \}$$

- Idea: If $\mathbf{S}(\mathbf{x})$ and $\mathbf{S}(\mathbf{y})$ **do not intersect**, then $\text{sim}(\mathbf{x}, \mathbf{y}) < t$ (why?)
- We can **ignore the prefix** of the vectors and index **only** the signature

Prefix-filtering example

- Data set and set up:
 - $D1 = \{(\text{jaguar}, 0.4), (\text{new}, 0.2), (\text{family}, 0.13), (\text{world}, 0.47)\}$
 - $D2 = \{(\text{jaguar}, 0.4), (\text{new}, 0.24)\}$
 - $D3 = \{(\text{jaguar}, 0.4), (\text{football}, 0.47), (\text{us}, 0.3)\}$
 - $m = \{(\text{jaguar}, 0.4), (\text{new}, 0.24), (\text{family}, 0.13), (\text{world}, 0.47), (\text{football}, 0.47), (\text{us}, 0.3)\}$
 - $\text{Dim} : \{(\text{jaguar}, 1), (\text{new}, 2), (\text{family}, 3), (\text{world}, 4), (\text{football}, 5), (\text{us}, 6)\}$
- Threshold $t = 0.2$:
 - $p(D1) = 1$:
 - $0.4 * 0.4 = 0.16 < t$
 - $0.4 * 0.4 + 0.2 * 0.24 = 0.208 > t$
 - $S(D1) = \{(\text{new}, 0.2), (\text{family}, 0.13), (\text{world}, 0.47)\}$

Prefix-filtering example

- Data set and set up:
 - $D1 = \{(\text{jaguar}, 0.4), (\text{new}, 0.2), (\text{family}, 0.13), (\text{world}, 0.47)\}$
 - $D2 = \{(\text{jaguar}, 0.4), (\text{new}, 0.24)\}$
 - $D3 = \{(\text{jaguar}, 0.4), (\text{football}, 0.47), (\text{us}, 0.3)\}$
 - $m = \{(\text{jaguar}, 0.4), (\text{new}, 0.24), (\text{family}, 0.13), (\text{world}, 0.47), (\text{football}, 0.47), (\text{us}, 0.3)\}$
 - Dim : $\{(\text{jaguar}, 1), (\text{new}, 2), (\text{family}, 3), (\text{world}, 4), (\text{football}, 5), (\text{us}, 6)\}$
- Threshold $t = 0.2$:
 - $p(D2) = 1$:
 - $0.4 * 0.4 = 0.16 < t$
 - $0.4 * 0.4 + 0.24 * 0.24 = 0.2176 > t$
 - $S(D2) = \{(\text{new}, 0.24)\}$

Prefix-filtering example

- Data set and set up:
 - $D1 = \{(\text{jaguar}, 0.4), (\text{new}, 0.2), (\text{family}, 0.13), (\text{world}, 0.47)\}$
 - $D2 = \{(\text{jaguar}, 0.4), (\text{new}, 0.24)\}$
 - $D3 = \{(\text{jaguar}, 0.4), (\text{football}, 0.47), (\text{us}, 0.3)\}$
 - $m = \{(\text{jaguar}, 0.4), (\text{new}, 0.24), (\text{family}, 0.13), (\text{world}, 0.47), (\text{football}, 0.47), (\text{us}, 0.3)\}$
 - $\text{Dim} : \{(\text{jaguar}, 1), (\text{new}, 2), (\text{family}, 3), (\text{world}, 4), (\text{football}, 5), (\text{us}, 6)\}$
- Threshold $t = 0.2$:
 - $p(D3) = 4$:
 - $0.4 * 0.4 = 0.16 < t$
 - $0.4 * 0.4 + 0.47 * 0.47 = 0.3809 > t$
 - $S(D3) = \{(\text{football}, 0.47), (\text{us}, 0.3)\}$

Prefix-filtering example

- Threshold $t = 0.2$:
 - $S(D1) = \{(\text{new}, 0.2), (\text{family}, 0.13), (\text{world}, 0.47)\}$
 - $S(D2) = \{(\text{new}, 0.24)\}$
 - $S(D3) = \{(\text{football}, 0.47), (\text{us}, 0.3)\}$
- Build the **inverted index** on signatures:
 - Candidate pair: $(D1, D2)$
 - Pruning pairs: $(D1, D3)$ and $(D2, D3)$ due to no intersection

Homework

- Implement the tfidf representations, inverted index and prefix-filtering algorithms with Python to solve the near-duplicate detection
 - Use the [KOS blog entries](https://archive.ics.uci.edu/ml/datasets/Bag+of+Words) data set from <https://archive.ics.uci.edu/ml/datasets/Bag+of+Words>
 - This data set consists of **3430** documents, each represented as a set of keywords. The total number of keywords of the corpus is **6960**
 - Finding all similar pair of documents whose inner products with tfidf representations are at least $t = 0.5$
 - **Hint:** if there are many similar pairs, you can increase the threshold t so that we have fewer candidate pairs